```
#include <stdio.h>
#include <stdlib.h>

// Definition of a Binary Tree Node
struct Node {
   int data;
   struct Node* left;
   struct Node* right;
};

// Function to create a new node
struct Node* createNode(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}

// Function to insert a node into the BST
struct Node* insert(struct Node* root, int value) {
   // If the tree/subtree is empty, create and return a new node
   if (root == NULL) {
      return createNode(value);
   }

   // Otherwise, recur down the tree
   if (value < root->data) {
      root->left = insert(root->left, value);
   } else if (value > root->data) {
      root->right = insert(root->right, value);
   }

   return root;
}

// Helper function to find the node with the minimum value in a subtree
struct Node* findMinValueNode(struct Node* node) {
   struct Node* current = node;
   // Loop down to find the leftmost leaf
   while (current && current->left != NULL) {
      current = current->left;
   }
   return current;
}

// Function to delete a node from the BST
struct Node* deleteNode(struct Node* root, int value) {
   if (root == NULL) {
      return root;
   }
```

```
    // Step 1: Traverse the tree to find the node to delete
    if (value < root->data) {
        root->left = deleteNode(root->left, value);
    } else if (value > root->data) {
        root->right = deleteNode(root->right, value);
    } else {
        // Found the node to be deleted! Handle the 3 cases:

        // Case 1 & 2: Node has no children (Leaf) or only one child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }

        // Case 3: Node has two children
        // Get the inorder successor (smallest node in the right subtree)
        struct Node* temp = findMinValueNode(root->right);

        // Copy the inorder successor's content to this node
        root->data = temp->data;

        // Delete the inorder successor
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

// Helper function to print tree in-order (Displays values in sorted order)
void inorderTraversal(struct Node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}

// Driver code to execute and demonstrate operations
int main() {
    struct Node* root = NULL;

    printf("--- Inserting Nodes: 50, 30, 20, 40, 70, 60, 80 ---\n");
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 20);
    insert(root, 40);
```

```
    insert(root, 70);
    insert(root, 60);
    insert(root, 80);

    printf("In-order tree traversal: ");
    inorderTraversal(root);
    printf("\n\n");

    printf("--- Deleting leaf node 20 ---\n");
    root = deleteNode(root, 20);
    printf("In-order tree traversal: ");
    inorderTraversal(root);
    printf("\n\n");

    printf("--- Deleting node 30 (has single child) ---\n");
    root = deleteNode(root, 30);
    printf("In-order tree traversal: ");
    inorderTraversal(root);
    printf("\n\n");

    printf("--- Deleting node 70 (has two children) ---\n");
    root = deleteNode(root, 70);
    printf("In-order tree traversal: ");
    inorderTraversal(root);
    printf("\n");

    return 0;
}
```